



UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

DIPARTIMENTO DI MATEMATICA

LAUREA TRIENNALE IN INFORMATICA

# **PROGETTO DI BASI DI DATI**

Basi di Dati di un Sistema Informativo per la Gestione di un torneo del GCC  
Pokémon

Alessandro Adami, Alberto José Toniolo

# Indice

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1 Abstract</b>                                                | <b>2</b>  |
| <b>2 Analisi dei Requisiti</b>                                   | <b>2</b>  |
| 2.1 Assunzione di Progetto . . . . .                             | 4         |
| 2.2 Diagramma E-R . . . . .                                      | 4         |
| <b>3 Progettazione Concettuale</b>                               | <b>4</b>  |
| <b>4 Progettazione Logica</b>                                    | <b>5</b>  |
| 4.1 Schema Logico . . . . .                                      | 6         |
| 4.2 Analisi delle Ridondanze . . . . .                           | 7         |
| 4.2.1 SCENARIO 1: CON RIDONDANZA . . . . .                       | 7         |
| 4.2.2 SCENARIO 2: SENZA RIDONDANZA . . . . .                     | 8         |
| 4.3 Eliminazioni delle generalizzazioni . . . . .                | 9         |
| 4.4 Schema Relazionale . . . . .                                 | 10        |
| <b>5 Implementazione in PostgreSQL e Definizione delle Query</b> | <b>11</b> |
| 5.1 Definizione delle Query . . . . .                            | 12        |
| 5.2 Creazione degli indici . . . . .                             | 14        |
| <b>6 Applicazione Software</b>                                   | <b>15</b> |

# 1 Abstract

Questo lavoro sviluppa una base di dati progettata per gestire in modo strutturato e coerente le informazioni relative a un torneo di un Gioco di Carte Collezionabili (TCG). Il sistema tiene traccia degli utenti partecipanti, dei mazzi utilizzati e della complessa gerarchia di carte che li compongono (suddivise in Pokémon, Energie e carte Trainer, con le relative abilità), nonché delle restrizioni di formato applicate e dell'ambiente di gioco, sia esso fisico o digitale.

Nel contesto del torneo, la base di dati distingue gli iscritti tra giocatori singoli e membri di una squadra, gestendo separatamente le informazioni relative all'acquisto del biglietto per i solitari e alle organizzazioni eSports che finanziano i team.

L'obiettivo principale del progetto è fornire un sistema gestionale completo che, oltre a registrare l'esito dei match, supporti le attività di analisi statistica (percentuale di utilizzo, percentuale di vittoria, ecc.) per singola carta. Questo permette agli organizzatori di monitorare le prestazioni dei partecipanti e di ricavare metriche fondamentali sul bilanciamento del gioco nel suo complesso.

## 2 Analisi dei Requisiti

Questa sezione riassume i requisiti a cui deve sottostare la base di dati per il torneo del Gioco di Carte Collezionabili (GCC).

**Ambiente di Gioco:** Ogni torneo si svolge in uno specifico ambiente, identificato univocamente e caratterizzato dalle seguenti informazioni:

- Un nome univoco che distingue l'ambiente.
- Il nome dell'organizzatore dell'evento.

Gli ambienti di gioco si suddividono in fisici e digitali.

**Ambienti Fisici:** Oltre alle informazioni generali, per gli ambienti fisici vengono registrati ulteriori dati:

- L'indirizzo della sede fisica in cui si tiene l'evento.
- La capienza massima di persone ammesse all'interno della struttura.

**Ambienti Digitali:** Per le piattaforme online, oltre alle informazioni di base, si registra:

- L'indirizzo IP del server di gioco.
- La regione geografica del server ospitante.

**Utenti:** I partecipanti al torneo sono identificati da un nickname univoco. Per ogni utente vengono registrate le seguenti informazioni:

- Il nickname che distingue in modo univoco ogni giocatore.
- L'indirizzo email e la data di iscrizione al sistema.
- L'ambiente di gioco a cui l'utente ha effettuato l'accesso.
- La collezione personale di carte, contenente le seguenti informazioni:
  - Il numero di copie possedute per ciascuna carta.
  - La lingua della carta.

Inoltre, per le regole del torneo, ogni utente può registrare e utilizzare **un solo mazzo**. Gli utenti si suddividono in: giocatori solitari e squadre.

**Giocatori Solitari:** Oltre alle informazioni generali, per i giocatori solitari si registra:

- Il nome e il cognome reale del partecipante.
- Il biglietto acquistato per l'iscrizione, caratterizzato dalle seguenti informazioni:

- Un codice seriale univoco.
- Il prezzo del biglietto.

**Squadre:** Per le squadre, oltre alle informazioni di base, si registra:

- Il tag identificativo della squadra.
- Il numero di membri attualmente iscritti nel team.
- L'eventuale organizzazione eSports affiliata con le seguenti informazioni:
  - La partita IVA dell'organizzazione.
  - Il nome della sede legale.
  - Il budget di sponsorizzazione destinato alla squadra.

Inoltre, per evitare conflitti di interesse all'interno del torneo, un'organizzazione eSports può finanziare **al massimo una singola squadra**.

**Mazzi e Carte:** Il gioco si basa sull'utilizzo di mazzi composti da carte da gioco.

- **Mazzo:** Ogni mazzo è identificato da un codice univoco, possiede un nome, una data di validazione ed è composto da un numero specifico di copie di determinate carte.
- **Carta:** Ogni carta è identificata da un codice alfanumerico univoco. Si registrano inoltre il nome, il testo di descrizione, un eventuale effetto speciale e l'espansione di provenienza (caratterizzata da codice, nome e data di rilascio).

Inoltre, una carta può essere inserita in specifiche liste di restrizione del torneo (banlist), di cui si memorizzano il nome della lista, la limitazione applicata e lo stato di ban. Le carte si suddividono in tre tipologie specifiche: Pokémon, Trainer e Energia.

**Carte Pokémon:** Oltre alle caratteristiche generali delle carte, per i Pokémon si registra:

- L'elemento del Pokémon, i Punti Salute (PS) e la fase evolutiva.
- La debolezza e il costo di ritirata.
- Le abilità possedute, caratterizzate dalle seguenti informazioni:
  - Il nome univoco dell'abilità.
  - La descrizione dell'effetto speciale associato.
  - I danni inflitti e il costo in energia richiesto per l'attivazione.

**Carte Trainer:** Per le carte Trainer, oltre alle informazioni di base, si registra:

- Il sottotipo della carta (es. Strumento, Aiuto, Stadio) e la durata dell'effetto.
- Il numero massimo di utilizzi consentiti durante la partita.

**Carte Energia:** Per le carte Energia, oltre alle informazioni di base, si registra:

- L'elemento specifico fornito dalla carta.
- Il tipo di bersaglio a cui l'energia può essere assegnata.

**Partite e Risultati:** Il torneo prevede lo svolgimento di incontri competitivi tra i giocatori iscritti.

- **Partita:** Rappresenta l'incontro intero disputato ed è identificata da un codice univoco. Si registrano l'ora di inizio e la fase del torneo in cui si svolge (es. Quarti di Finale).
- **Risultato:** Al termine di ogni partita, per ciascun utente partecipante viene generato un referto che registra il punteggio finale ottenuto, gli eventuali bonus assegnati e le penalità ricevute durante l'incontro.

La presente base di dati è progettata per gestire il ciclo di vita di un singolo torneo alla volta. Pertanto, le informazioni generali sull'evento (es. nome del torneo) sono considerate implicite nel contesto e non richiedono un'entità dedicata. Il torneo è comunque diviso tra fisico e digitale, perciò risulta coerente tenere conto dell'ambiente di gioco.

Il diagramma ER illustra la struttura di un sistema di gioco online. Le entità principali e le loro attributi sono:

- Carta**: Nome\_Carta, Testo\_Descrizione, Effetto, Codice\_Carta, Lingua, Numero\_Copie.
- Pokemon**: Elemento\_Pokemon, Punti\_Salute, Fase\_Evolutiva, Costo\_Ritratta, Debolezza.
- Trainer**: Massimo\_Utilizzi, Durata, Scettro.
- Partita**: Ora\_Inizio, Fase\_Torneo, Codice\_Partita.
- Squadra**: Tag\_Squadra, Numero\_Membri.
- Utente**: Nickname, Email, Data\_Iscrizione.
- Ambiente\_Gioco**: Nome\_Ambiente, Organizzatore.
- Organizzazione Esports**: Partita\_NA, Sede\_Legale, Budget\_Sponsorizzazione.
- Partita**: Ora\_Inizio, Fase\_Torneo, Codice\_Partita.
- Expansione**: Codice\_Expansione, Nome\_Expansione, Data\_Rilascio.
- Restrizione**: Nome\_Lista, Limitazione, Bannato.
- Energia**: Bersaglio, Elemento.

Le relazioni e le loro cardinalità sono:

- Concorrenza** (Pokemon - Carta): (1,N).
- Pubblicazione** (Carta - Espansione): (1,N).
- Selezione** (Carta - Restrizione): (0,N).
- Collezione** (Carta - Utente): (1,N).
- Acquisto** (Carta - Mazzo): (1,N).
- Finanziata** (Squadra - Organizzazione Esports): (0,1).
- Pagamento** (Squadra - Organizzazione Esports): (1,N).

Il diagramma include anche relazioni di tipo "has" (indicate da frecce piene) e "owns" (indicate da linee con un cerchio alla fine).

### 3 Progettazione Concettuale

Per gestire entità con attributi parzialmente condivisi, il modello sfrutta tre gerarchie principali:

- La relazione tra *Utente* e *Mazzo* è stata vincolata con cardinalità 1:1, forzando l'uso di un unico mazzo per partecipante.
- La relazione tra *Organizzazione Esports* e *Squadra* presenta una cardinalità massima 1:1, quindi una Organizzazione sponsorizza solamente una Squadra; viceversa una Squadra è sponsorizzata da una sola Organizzazione, in quanto ne funge da rappresentante.

La natura combinatoria del gioco di carte ha richiesto l'uso di relazione N:M, a cui sono stati assegnati attributi propri:

- Le relazioni di appartenenza (*Collezione* e *PartOf*) includono attributi di quantità, permettendo il tracciamento delle copie.
- La relazione *Corrispondenza* tra *Pokémon* e *Abilità* permette all'abilità di appartenere a Pokémon diversi e di non essere esclusive.

Nei Giochi di Carte Collezionabili è frequente che più partecipanti decidano di competere presentando la stessa identica lista di carte, pur possedendo materialmente copie fisiche o digitali distinte. Per evitare una grande ridondanza dei dati, che si verificherebbe duplicando l'elenco delle carte per ogni singolo giocatore, l'entità *Mazzo* è stata interpretata come "lista/decklist" e non come oggetto fisico univoco.

Abbiamo inoltre deciso di promuovere a entità debole il concetto di *Risultato* e di identificarla tramite la relazione *Termine* con *Partita* e *Ottenimento* con *Utente*, invece che mantenerla come relazione N:M tra le due entità che la circondano. Ciò alleggerisce il modello e lo rende predisposto a future espansioni, come l'aggiunta di un log dettagliato dei turni.

La tabella 1 a pagina 6 riassume tutte le entità e relazioni individuate dalla analisi dei requisiti nel diagramma E-R, con i relativi attributi rilevanti dall'analisi. Per le entità, viene anche fornito l'identificatore, che contiene due relazioni per l'entità Risultato.

Il presente schema E-R non permette di rappresentare direttamente i seguenti vincoli:

- L'attributo *Costo\_Energia* in *Abilità* non può essere minore di 0

$$\forall A \in \text{Abilità}, \quad A.\text{Costo\_Energia} \geq 0$$

- Per ogni partita, il numero minimo di giocatori è 1 mentre il massimo è 2, ovvero:

$$\forall P \in \text{Partita}, \quad \exists U \in \text{Utente} : \quad 1 \leq |U| \leq 2$$

## 4 Progettazione Logica

In questa sezione viene illustrato il processo di "traduzione" dello schema concettuale in uno schema logico, con l'obiettivo di rappresentare i dati in modo preciso ed efficiente. Il primo passo consiste nell'analizzare le eventuali ridondanze nel modello, al fine di ottimizzare la struttura complessiva. Successivamente, si procede con l'eliminazione delle tre generalizzazioni. Infine, viene presentato il diagramma ristrutturato, con una descrizione delle modifiche apportate.

## 4.1 Schema Logico

| Entità                 | Descrizione                                                      | Attributi                                                                        | Identificatore                               |
|------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------|
| Carta                  | Carta Generica                                                   | <i>Codice_Carta, Nome_Carta, Testo_Descrizione, Effetto</i>                      | <i>Codice_Carta</i>                          |
| Mazzo                  | Check-list di un mazzo di carte                                  | <i>Codice_Mazzo, Nome_Mazzo, Data_Validazione, Numero_Carte</i>                  | <i>Codice_Mazzo</i>                          |
| Espansione             | Espansione di provenienza delle carte                            | <i>Codice_Espansione, Nome_Espansione, Data_Rilascio</i>                         | <i>Codice_Espansione, Nome_Espansione</i>    |
| Restrizione            | Restrizione dell'uso di una carta                                | <i>Nome_Lista, Limitazione, Bannato</i>                                          | <i>Nome_Lista</i>                            |
| Pokemon                | Carta Pokémon                                                    | <i>Elemento_Pokemon, Punti_Salute, Fase_Evolutiva, Costo_Ritirata, Debolezza</i> |                                              |
| Abilità                | Abilità di un Pokémon                                            | <i>Nome_Abilità, Descrizione_Effetto, Danni, Costo_Energia</i>                   | <i>Nome_Abilità</i>                          |
| Energia                | Carta Energia                                                    | <i>Bersaglio, Elemento</i>                                                       |                                              |
| Trainer                | Carta Trainer                                                    | <i>Massimo_Utilizzi, Durata, Sottotipo</i>                                       |                                              |
| Utente                 | Utente che gioca                                                 | <i>Nickname, Email, Data_Iscrizione</i>                                          | <i>Nickname</i>                              |
| Squadra                | Squadra di utenti                                                | <i>Tag_Squadra, Numero_Membri</i>                                                |                                              |
| Organizzazione Esports | Organizzazione per squadre                                       | <i>Partita_IVA, Sede_Legale, Budget_Sponsorizzazione</i>                         | <i>Partita_IVA</i>                           |
| Solitario              | Utente che gioca da solo                                         | <i>Nome_Reale, Cognome_Reale</i>                                                 |                                              |
| Biglietto              | Biglietto d'iscrizione al torneo                                 | <i>Codice_Seriale, Prezzo</i>                                                    | <i>Codice_Seriale</i>                        |
| Risultato              | Risultato alla fine di una partita                               | <i>Punteggio, Bonus_Assignati, Penalità_Assignate</i>                            | Relazione "Termine", Relazione "Ottenimento" |
| Partita                | Incontro disputato tra due giocatori durante una fase del torneo | <i>Codice_Partita, Ora_Inizio, Fase_Torneo</i>                                   | <i>Codice_Partita</i>                        |
| Ambiente_Gioco         | Ambiente in cui si svolge il torneo                              | <i>Nome_Ambiente, Organizzatore</i>                                              | <i>Nome_Ambiente</i>                         |
| Fisico                 | Torneo in un posto fisico                                        | <i>Indirizzo_Sede, Capacità_Massima</i>                                          |                                              |
| Digitale               | Torneo su una piattaforma Online                                 | <i>Indirizzo_IP, Regione_Server</i>                                              |                                              |

(a) Entità

Tabella 1: Dizionario dei dati.

| Relazione      | Descrizione                                                      | Componente                      | Attributi                   |
|----------------|------------------------------------------------------------------|---------------------------------|-----------------------------|
| Corrispondenza | Corrispondenza tra un Pokémon e la sua abilità pos-seduta        | Pokemon, Abilità                |                             |
| PartOf         | Appartenenza di una carta ad un mazzo                            | Carta, Mazzo                    | <i>Quantità</i>             |
| Selezione      | Carta selezionata nella li-sta per la restrizione                | Carta, Restrizione              |                             |
| Collezione     | Carta appartenente alla collezione dell'utente                   | Utente, Carta                   | <i>Lingua, Numero_Copie</i> |
| Creazione      | Creazione di un mazzo da parte di un utente                      | Utente, Mazzo                   |                             |
| Finanziata     | Organizzazione Esports fi-nanzia la squadra                      | Squadra, Organizzazione Esports |                             |
| Pagamento      | Utente singolo paga il bi-glietto di iscrizione                  | Solitario, Biglietto            |                             |
| Ottenimento    | Utente ottiene un risultato a fine match                         | Utente, Risultato               |                             |
| Termine        | Al termine di una partita si ottiene un risultato                | Risultato, Partita              |                             |
| Accesso        | Utente accede al torneo che si realizza in un am-biente di gioco | Utente, Ambiente_Gioco          |                             |

(b) Relazioni

Tabella 1: Dizionario dei dati (continua).

## 4.2 Analisi delle Ridondanze

L'attributo *Numero\_Carte* nella tabella Mazzo memorizza il numero totale di carte contenute in un mazzo, introducendo una ridondanza. Tale informazione, infatti, potrebbe essere ricavata in modo dinamico calcolando la somma dell'attributo *Quantità* nella relazione PartOf per un determinato mazzo.

L'analisi seguente valuta l'opportunità di mantenere o eliminare tale ridondanza basandosi sul bilancio dei costi di due operazioni principali:

- **Operazione 1:** Inserire o rimuovere una carta all'interno di un mazzo (Frequenza: **1000 volte/giorno**)
- **Operazione 2:** Verificare la validità di un mazzo (presenza di esattamente 60 carte) prima di iniziare una partita (Frequenza: **200 volte/giorno**).

### Volumi della Base di Dati Assunti

| Concetto | Costrutto | Volume |
|----------|-----------|--------|
| Mazzo    | E         | 100    |
| PartOf   | R         | 5000   |
| Carta    | E         | 5000   |

Tabella 2: Assunzione dei volumi nella base di dati

### 4.2.1 SCENARIO 1: CON RIDONDANZA

- **Operazione 1: Inserimento/Rimozione carta nel mazzo**



| Concetto | Costrutto | Accessi | Tipo |                     |
|----------|-----------|---------|------|---------------------|
| Carta    | E         | 1       | L    | x 1000 volte/giorno |
| PartOf   | R         | 1       | S    | x 1000 volte/giorno |
| Mazzo    | E         | 1       | L    | x 1000 volte/giorno |
| Mazzo    | E         | 1       | S    | x 1000 volte/giorno |

Tabella 3: Accessi causati da Inserimento/Rimozione

**Nota:** nell'entità Carta non viene fatta una scrittura quando viene inserita in Mazzo perché le carte sono già salvate nella base di dati, perciò avviene solo una lettura.

- **Operazione 2: Verifica validità del mazzo** (Frequenza: **200 volte/giorno**)

| Concetto | Costrutto | Accessi | Tipo |                    |
|----------|-----------|---------|------|--------------------|
| Mazzo    | E         | 1       | L    | x 200 volte/giorno |
| PartOf   | R         | 1       | L    | x 200 volte/giorno |

Tabella 4: Accessi causati dalla verifica della validità del mazzo

#### Calcolo del Costo Totale (Con Ridondanza)

Assumendo, come da convenzione, che un accesso in scrittura pesi il doppio di un accesso in lettura:

- **Costo Op.1:**  $1000 * 2 + 1000 * 2 * 2 = 6000$
- **Costo Op.2:**  $200 * 2 = 400$

**Costo totale con ridondanza:**  $6000 + 400 = 6400$

#### 4.2.2 SCENARIO 2: SENZA RIDONDANZA

**Nota:** Il numero medio di righe distinte della relazione PartOf associate a ogni singolo mazzo è pari a:

$$\frac{Volume(PartOf)}{Volume(Mazzo)} = \frac{5000}{100} = 50 \text{ righe medie per mazzo in PartOf}$$

- **Operazione 1: Inserimento/Rimozione carta nel mazzo** (Frequenza: **1000 volte/giorno**)

| Concetto | Costrutto | Accessi | Tipo |                     |
|----------|-----------|---------|------|---------------------|
| Carta    | E         | 1       | L    | x 1000 volte/giorno |
| PartOf   | R         | 1       | S    | x 1000 volte/giorno |

Tabella 5: Accessi causati da Inserimento/Rimozione

- **Operazione 2: Verifica validità mazzo** (Frequenza: **200 volte/giorno**)

| Concetto | Costrutto | Accessi | Tipo |                    |
|----------|-----------|---------|------|--------------------|
| Mazzo    | E         | 1       | L    | x 200 volte/giorno |
| PartOf   | R         | 50      | L    | x 200 volte/giorno |

Tabella 6: Accessi causati dalla verifica della validità del mazzo

### Calcolo del Costo Totale (Senza Ridondanza)

Assumendo, come da convenzione, che un accesso in scrittura pesi il doppio di un accesso in lettura:

- **Costo Op.1:**  $1000 + 1000 * 2 = 3000$
- **Costo Op.2:**  $200 + 200 * 50 = 10200$

**Costo totale senza ridondanza:**  $3000 + 10200 = 13200$

Abbiamo quindi:

$$\frac{\text{CostoConRidondanza}}{\text{CostoSenzaRidondanza}} = \frac{6400}{13200} = 48\%$$

Il confronto tra i due schemi evidenzia che, sebbene l'assenza di ridondanza alleggerisca l'operazione di scrittura (Operazione 1), essa rende insostenibile l'operazione di lettura prima di ogni partita (Operazione 2), la quale richiederebbe un conteggio aggregato iterato su circa 50 righe della tabella di giunzione per ogni singolo match avviato. L'analisi suggerisce quindi di mantenere l'attributo ridondante, ottimizzando il numero di accessi di circa il 50%.

### 4.3 Eliminazioni delle generalizzazioni

Le generalizzazioni descritte in Sezione 3 vengono eliminate attraverso una ristrutturazione dello schema concettuale, con l'obiettivo di semplificare la successiva implementazione del modello relazionale e ridurre la presenza di valori nulli. Le modifiche vengono applicate come segue:

- **CARTA:** La generalizzazione di Carta viene rimossa includendo le relazioni Is-Poke, Is-Ene e Is-Train, come mostrato in figura 2. Questa scelta riduce la presenza di valori nulli: se si mantenesse un'unica entità Carta con tutti gli attributi relativi ai tre possibili tipi di una carta, avremmo diversi campi nulli, come i *Punti\_Salute*, *Fase\_Evolutiva*, ecc., mentre ci stiamo riferendo a una carta Trainer (e così anche per gli altri tipi). Separando le informazioni con relazioni dedicate, si garantisce che ogni entità contenga solo gli attributi rilevanti. In questa soluzione, coerentemente con la metodologia vista a lezione, gli identificatori di Energia, Pokémon e Trainer coincidono con le rispettive relazioni con Carta. Sarebbe stato possibile rimuovere l'entità Carta, incorporandone gli attributi nelle entità figlie. La soluzione sarebbe stata del tutto legittima, ma avrebbe causato la necessità di duplicare la relazione Part-Of e le altre relazioni annesse verso Energia, Pokémon e Trainer.
- **AMBIENTE GIOCO:** In maniera analoga alla generalizzazione di carta, per Ambiente\_Gioco la generalizzazione viene rimossa introducendo le relazioni Is-Fis e Is-Dig. Nello stesso modo mantenendo un'unica entità avremmo valori nulli come *Indirizzo\_Ip* e *Regione\_Server* quando il contesto del torneo è fisico (e viceversa). Sarebbe stato possibile rimuovere l'entità Ambiente\_Gioco incorporando gli attributi nelle entità figlie, ma avrebbe anche qui causato la necessità di duplicare la relazione Accesso in due relazioni.
- **UTENTE:** Anche per Utente la generalizzazione viene rimossa introducendo due relazioni, Is-Squad e Is-Solo. Unendo tutti gli attributi sotto una singola entità Utente porterebbe a campi nulli come *Tag\_Squadra* e *Numero\_Membri* quando ci si riferisce a un utente solitario (e viceversa). Analogamente agli altri due casi, sarebbe stato possibile incorporare gli attributi nelle unità figlie, ma anche qui avremmo duplicato tutte le relazioni annesse, come Accesso e Ottenimento, verso Squadra e Solitario.



- Solitario.Biglietto → Biglietto.Codice\_Seriale
- **Organizzazione\_Esports** (Partita\_IVA, Sede\_Legale, Budget\_Sponsorizzazione)
- **Squadra** (Utente, Tag\_Squadra, Numero\_Membri, Esports\*)
  - Squadra.Utente → Utente.Nickname
  - Squadra.Esports → Organizzazione\_Esports.Partita\_IVA
- **Carta** (Codice\_Carta, Nome\_Carta, Testo\_Descrizione, Effetto\*, Codice\_Espansione, Nome\_Espansione, Restrizione\*)
  - Carta.(Codice\_Espansione, Nome\_Espansione) → Espansione.(Codice\_Espansione, Nome\_Espansione)
  - Carta.Restrizione → Restrizione.Nome\_Lista
- **Collezione** (Carta, Utente, Numero\_Copie, Lingua)
  - Collezione.Carta → Carta.Codice\_Carta
  - Collezione.Utente → Utente.Nickname
- **PartOf** (Carta, Mazzo, Quantità)
  - PartOf.Carta → Carta.Codice\_Carta
  - PartOf.Mazzo → Mazzo.Codice\_Mazzo
- **Pokemon** (Carta, Elemento\_Pokemon, Punti\_Salute, Fase\_Evolutiva, Debolezza, Costo\_Ritirata)
  - Pokemon.Carta → Carta.Codice\_Carta
- **Trainer** (Carta, Massimo\_Utilizzi\*, Sottotipo, Durata)
  - Trainer.Carta → Carta.Codice\_Carta
- **Energia** (Carta, Bersaglio, Elemento)
  - Energia.Carta → Carta.Codice\_Carta
- **Abilità** (Nome\_Abilità, Descrizione\_Effetto, Danni\*, Costo\_Energia\*)
- **Partita** (Codice\_Partita, Ora\_Inizio\*, Fase\_Torneo)
- **Risultato** (Utente, Partita, Punteggio, Bonus\_Assegnati\*, Penalità\_Assegnate\*)
  - Risultato.Utente → Utente.Nickname
  - Risultato.Partita → Partita.Codice\_Partita
- **Corrispondenza** (Pokemon, Abilità)
  - Corrispondenza.Pokemon → Pokemon.Carta
  - Corrispondenza.Abilità → Abilità.Nome\_Abilità

## 5 Implementazione in PostgreSQL e Definizione delle Query

Il file *collezionismo.sql* contiene il codice SQL necessario per la creazione e il popolamento delle tabelle del database. Questo file include inoltre una serie di Query per l'estrazione dei dati e un indice creato specificamente per migliorare le prestazioni di una di queste interrogazioni.

## 5.1 Definizione delle Query

**Query 1:** Elencare a che espansione appartengono le carte più giocate.

```
1 SELECT C.Nome_Carta, E.Nome_Espansione, COUNT(DISTINCT M.Codice_Mazzo) AS Conteggio
2 FROM Espansione E
3 JOIN Carta C ON E.Codice_Espansione = C.Codice_Espansione AND E.Nome_Espansione = C.
  Nome_Espansione
4 JOIN PartOf P ON C.Codice_Carta = P.Carta
5 JOIN Mazzo M ON P.Mazzo = M.Codice_Mazzo
6 JOIN Utente U ON U.Mazzo = M.Codice_Mazzo
7 GROUP BY C.Nome_Carta, E.Nome_Espansione
8 HAVING COUNT(DISTINCT M.Codice_Mazzo) >= 3
9 ORDER BY Conteggio DESC;
```

Estratto dell'output:

|   | nome_carta<br>character varying (20) 🔒 | nome_espansione<br>character varying (20) 🔒 | conteggio<br>bigint 🔒 |
|---|----------------------------------------|---------------------------------------------|-----------------------|
| 1 | Arbitro                                | Evoluzioni a Paldea                         | 3                     |
| 2 | Caramella Rara                         | Scarlatto e Violetto                        | 3                     |
| 3 | Ricerca Accademica                     | Scarlatto e Violetto                        | 3                     |

Figura 3: Risultato della Query 1.

**Query 2:** Elencare quali giocatori hanno disputato più di 5 partite totali in ambienti digitali ottenendo una media di punteggio superiore a 2 punti. Vengono attribuiti 0 punti per la sconfitta, 1 per il pareggio e 3 per la vittoria; 2 punti implicano un numero predominante di vittorie.

```
1 SELECT U.Nickname, COUNT(R.Partita) AS Partite_Digitali, AVG(R.Punteggio) AS
  Punteggio_Medio
2 FROM Utente U
3 JOIN Digitale D ON U.Ambiente = D.Ambiente
4 JOIN Risultato R ON R.Utente = U.Nickname
5 GROUP BY U.Nickname
6 HAVING COUNT(R.Partita) > 5 AND AVG(R.Punteggio) > 2.0
7 ORDER BY Punteggio_Medio DESC;
```

Estratto dell'output:

|   | nickname<br>[PK] character varying (20) ✎ | partite_digitali<br>bigint 🔒 | punteggio_medio<br>numeric 🔒 |
|---|-------------------------------------------|------------------------------|------------------------------|
| 1 | SnorlaxFan                                | 6                            | 2.6666666666666667           |
| 2 | ProGamer_IT                               | 7                            | 2.5714285714285714           |
| 3 | IceKing                                   | 6                            | 2.5000000000000000           |

Figura 4: Risultato della Query 2.

**Query 3:** Elencare quali mazzi nel database presentano un'incoerenza tra il valore ridondante *Numero\_Carte* e il conteggio effettivo delle carte fisicamente presenti nella tabella PartOf

```
1 SELECT M.Codice_Mazzo, M.Nome_Mazzo, M.Numero_Carte AS Valore_Ridondanza, SUM(P.Quantita)
   AS Valore_Quantita
2 FROM Mazzo M JOIN PartOf P ON M.Codice_Mazzo = P.Mazzo
3 GROUP BY M.Codice_Mazzo, M.Nome_Mazzo, M.Numero_Carte
4 HAVING M.Numero_Carte <> SUM(P.Quantita);
```

**Estratto dell'output:**

|   | codice_mazzo<br>[PK] character varying (8) | nome_mazzo<br>character varying (15) | valore_atteso<br>integer | valore_quantità<br>bigint |
|---|--------------------------------------------|--------------------------------------|--------------------------|---------------------------|
| 1 | MZZ-0005                                   | Mew VMAX                             | 60                       | 2                         |
| 2 | MZZ-0002                                   | Lost Zone Box                        | 60                       | 7                         |
| 3 | MZZ-0001                                   | Charizard ex                         | 60                       | 11                        |
| 4 | MZZ-0006                                   | Gardevoir ex                         | 60                       | 17                        |
| 5 | MZZ-0007                                   | Chien-Pao                            | 60                       | 8                         |
| 6 | MZZ-0004                                   | Snorlax Stall                        | 60                       | 3                         |
| 7 | MZZ-0003                                   | Lugia VSTAR                          | 60                       | 4                         |

Figura 5: Risultato della Query 3.

**Query 4:** Creiamo una vista permanente che calcoli la classifica live del torneo (*nickname*, somma dei punti e partite disputate). Dopodiché, interroghiamo la vista per estrarre solo la i primi tre giocatori e il rispettivo punteggio.

```
1 CREATE VIEW Classifica AS (
2   SELECT Utente, SUM(Punteggio) AS Punti_Totali, COUNT(Partita) AS Partite_Giocate
3   FROM Risultato
4   GROUP BY Utente
5 );
6 SELECT Utente, Punti_Totali
7 FROM Classifica
8 ORDER BY Punti_Totali DESC
9 LIMIT 3;
```

**Estratto dell'output:**

|   | utente<br>character varying (20) | punti_totali<br>bigint |
|---|----------------------------------|------------------------|
| 1 | ProGamer_IT                      | 18                     |
| 2 | SnorlaxFan                       | 16                     |
| 3 | IceKing                          | 15                     |

Figura 6: Risultato della Query 4.

**Query 5:** Trova i dettagli di tutti i giocatori (*Nome, Cognome, Email*) che hanno utilizzato almeno una volta un ambiente di gioco "Fisico" (ovvero che hanno giocato un match dal vivo, escludendo chi ha giocato solo online).

```

1 SELECT S.Nome_Reale, S.Cognome_Reale, U.Email
2 FROM Solitario S JOIN Utente U ON S.Utente = U.Nickname
3 WHERE U.Nickname IN (
4     SELECT R.Utente
5     FROM Risultato R
6     JOIN Utente U ON R.Utente = U.Nickname
7     JOIN Fisico F ON U.Ambiente = F.Ambiente
8 );

```

Estratto dell'output:

|   | nome_reale<br>character varying (20) 🔒 | cognome_reale<br>character varying (10) 🔒 | email<br>character varying (20) 🔒 |
|---|----------------------------------------|-------------------------------------------|-----------------------------------|
| 1 | Satoshi                                | Tajiri                                    | ash@poke.com                      |
| 2 | Mario                                  | Rossi                                     | master@mail.com                   |
| 3 | Giovanni                               | Rocket                                    | mew@mail.com                      |
| 4 | Sofia                                  | Bianchi                                   | garde@mail.it                     |

Figura 7: Risultato della Query 5.

## 5.2 Creazione degli indici

L'analisi del piano di ottimizzazione è stata tarata sulle specifiche scelte di design della base di dati, ottimizzando quattro delle query tramite l'introduzione di due indici mirati.

### 1. Ottimizzazione delle Query 1 e 3 (Tabella PartOf)

Per ottimizzare le Query 1 e 3 si è scelto di implementare il seguente indice:

```

1 CREATE INDEX idx_partof_mazzo ON PartOf (Mazzo);

```

La tabella PartOf presenta una chiave primaria composta definita come PRIMARY KEY (Carta, Mazzo). Poiché l'attributo Carta compare per primo, l'indice default della chiave primaria non può essere sfruttato dal DBMS per query che filtrano o uniscono i dati basandosi unicamente sul campo Mazzo. L'indice B+ Tree manuale idx\_partof\_mazzo colma questa lacuna, fornendo una struttura ordinata che accelera le operazioni di Equi-Join e le successive fasi di raggruppamento.

### 1. Ottimizzazione delle Query 2 e 4 (Tabella Risultato)

Per le operazioni di classificazione e calcolo delle medie delle Query 2 e 4, si è adottata un Indice su attributi multipli, ovvero il seguente:

```

1 CREATE INDEX idx_risultato ON Risultato(Utente, Punteggio);

```

Questo indice memorizza in modo combinato l'attributo di raggruppamento (Utente) e l'attributo di aggregazione (Punteggio). Durante l'esecuzione delle Query 2 e 4, il motore di PostgreSQL non avrà bisogno di accedere alle pagine di dati della tabella sul disco, ma potrà calcolare le funzioni AVG(Punteggio) e SUM(Punteggio) eseguendo un Index-Only Scan direttamente all'interno dell'indice. Questo approccio azzerà i tempi di I/O sul disco, massimizzando l'efficienza della RAM.

## 6 Applicazione Software

Il file *query.c* contiene il codice C necessario per connettersi al database e visualizzare a schermo i risultati delle query presentate nella Sezione 5. Una volta compilato ed eseguito il programma tramite il comando:

```
gcc query.c -o query -lpq && ./query
```

Il programma ritornerà il risultato delle 5 Query descritte in precedenza:

- **Query 1:** A che espansione appartengono le carte più giocate.
- **Query 2:** Quali giocatori hanno disputato più di  $N$  partite totali in ambienti digitali ottenendo una media di punteggio superiore a  $M$  punti. La Query è stata resa parametrica, lasciando  $N$  e  $M$  a scelta dell'utente al momento dell'esecuzione.
- **Query 3:** Quali mazzi nel database presentano un'incoerenza tra il valore ridondante *Numero\_Carte* e il conteggio effettivo delle carte fisicamente presenti nella tabella *PartOf*.
- **Query 4:** I primi tre giocatori e il rispettivo punteggio.
- **Query 5:** Tutti i giocatori (*Nome*, *Cognome*, *Email*) che hanno utilizzato almeno una volta un ambiente di gioco "Fisico" (ovvero che hanno giocato un match dal vivo, escludendo chi ha giocato solo online),

separate da una riga di delimitazione.